

C PROGRAMMING COMPREHENSIVE REFERENCE MANUAL

A complete tabular synthesis mapping source code implementations, runtime output behaviors, and foundational syntactic analysis across Chapters 1-11.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 1 1. Hello World Program <pre>#include<stdio.h> int main() { printf("Hello World"); return 0; }</pre>	Hello World	<code>#include<stdio.h></code> is a preprocessor directive that imports the Standard Input Output library needed for <code>printf()</code>. <code>int main()</code> is the mandatory entry point for every C program. <code>printf()</code> is a standard library function used to display formatted text on the screen. <code>return 0;</code> terminates the main function and signals a successful execution to the operating system.
CHAPTER 1 2. Variable Declaration <pre>#include<stdio.h> int main() { int number; int age; int price; return 0; }</pre>	(No visual output)	- Demonstrates how memory space is reserved for storing data. <code>int</code> is a keyword representing the integer data type. <code>number</code>, <code>age</code>, and <code>price</code> are identifiers (variable names) that will hold whole numbers.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 1 3. Variable Initialization <pre>#include<stdio.h> int main() { int age = 22; float pi = 3.14; char percentage = '%'; return 0; }</pre>	(No visual output)	- Demonstrates allocating and populating memory simultaneously using the assignment operator (=). - Introduces three basic data types: <code>int</code> for integers (e.g., 22), <code>float</code> for real or fractional numbers (e.g., 3.14), and <code>char</code> for individual single-byte characters. - Character values must always be enclosed in single quotes (e.g., <code>'%'</code>).
CHAPTER 1 4. Single-Line Comments <pre>#include<stdio.h> //This program prints Hello World int main() { printf("Hello World"); return 0; }</pre>	Hello World	- Introduces internal documentation via <code>//</code> , which denotes a single-line comment. - Comments are entirely stripped away by the compiler and do not affect the compiled binary size or program performance. They are purely for human understanding.
CHAPTER 1 5. Output with Format Specifiers <pre>#include<stdio.h> int main() { int age = 22; float pi = 3.14; char percentage = '%'; printf("age is %d", age); printf("age is %f", pi); printf("age is %c", percentage); return 0; }</pre>	age is 22age is 3.140000age is %	- Demonstrates format specifiers inside placeholders: <code>%d</code> for integers, <code>%f</code> for floating-point values, and <code>%c</code> for characters. - Notice that the text outputs run together continuously because no newline indicators (<code>\n</code>) were included in the formatting strings. - By default, <code>%f</code> prints up to 6 decimal places.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 1 6. Standard Input & Arithmetic Expression <pre>#include<stdio.h> int main() { int a, b; printf("enter a \n"); scanf("%d", &a); printf("enter b \n"); scanf("%d", &b); printf("sum of a & b is : %d \n", a+b); return 0; }</pre>	<pre>enter a 5 (User input) enter b 10 (User input) sum of a & b is : 15</pre>	• <code>scanf()</code> is the function used to read values from the terminal keyboard. • The ampersand symbol (&) represents the "Address-of" operator. It tells <code>scanf()</code> exactly where in the computer memory the user's input value should be placed. • <code>\n</code> is an escape sequence that forces a newline. • Arithmetic operations (<code>a+b</code>) can be computed dynamically straight inside a <code>printf()</code> call.
CHAPTER 1 7. Practice Q1: Area of a Square <pre>#include<stdio.h> //area of square int main() { int side; scanf("%d", &side); printf("%d", side * side); return 0; }</pre>	<pre>4 (User input) 16</pre>	• Practical application combining single-integer input and immediate mathematical calculation. • The area expression <code>side * side</code> evaluates instantly, and its outcome is printed without assigning the result to an auxiliary variable first, saving memory overhead.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 1 8. Practice Q2: Area of a Circle <pre>#include<stdio.h> //area of square int main() { float radius; scanf("%f", &radius); printf("%f", 3.14 * radius * radius); return 0; }</pre>	<pre>2.0 (User input) 12.560000</pre>	• Demonstrates working with continuous physical properties (radius) using floating-point decimals. • Uses <code>%f</code> across both input parsing and output composition. • Demonstrates mixing literal decimals (<code>3.14</code>) with floating variables inside numeric expressions.
CHAPTER 2 1. Type Declaration Instructions <pre>#include<stdio.h> int main() { int age = 22; int oldAge = age; int newAge = oldAge + 2; printf("new age is : %d", newAge); int rupee = 1, dollar; dollar = 74; // valid declaration int age1, age2, age3; age1 = age2 = age3 = 22; return 0; }</pre>	<pre>new age is : 24</pre>	• Rule of Order: Variables must always be declared before they can be evaluated or assigned. The commented out block in the source code highlights that evaluating equations with variables declared further down will prompt compile errors. • Demonstrates multiple variable chains where assignments resolve right-to-left: <code>age1 = age2 = age3 = 22;</code> .

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 2 2. Arithmetic Instructions & Modulo <pre>#include<stdio.h> int main() { int a = 1, b = 2, c = 3; a = b + c; printf("%d \n", 3 % 2); printf("%d \n", -3 % 2); return 0; }</pre>	<pre>1 -1</pre>	• Assignment Symmetry: The single target variable must always reside exclusively on the left side of the operator (<code>a = b + c</code> is completely valid, whereas <code>b + c = a</code> throws an error). • Introduces the Modulo operator (%), which returns the remainder of integer division. • Sign Rule: In C, the sign of a modulo outcome is defined strictly by the sign of the numerator (the first operand). Thus, <code>-3 % 2</code> evaluates to <code>-1</code>.
CHAPTER 2 3. Implicit Type Conversion <pre>#include<stdio.h> int main() { printf("sum of 2 & 3 : %d\n", 2 + 3); printf("sum of 2.0 & 3 : %f\n", 2.0 + 3); printf("sum of 2.0 & 3.0 : %f\n", 2.0 + 3.0); return 0; }</pre>	<pre>sum of 2 & 3 : 5 sum of 2.0 & 3 : 5.000000 sum of 2.0 & 3.0 : 5.000000</pre>	• Demonstrates type casting hierarchies: • <code>int</code> operated with an <code>int</code> returns an <code>int</code>. • <code>float</code> operated with an <code>int</code> forces an implicit promotion of the integer, returning a <code>float</code>. • <code>float</code> operated with a <code>float</code> returns a <code>float</code>.
CHAPTER 2 4. Precedence & Associativity <pre>#include<stdio.h> int main() { printf(" Output : %d", 5+2/2*3); return 0; }</pre>	<pre>Output : 8</pre>	• When an expression lacks parenthetical boundaries, operator hierarchies govern the execution order. • Multiplication (<code>*</code>) and Division (<code>/</code>) enjoy higher priority ranks than Addition (<code>+</code>). • Since division and multiplication hold the exact same priority rank, the compiler resolves them via Left-to-Right Associativity: First <code>2/2 = 1</code>, next <code>1*3 = 3</code>, and lastly <code>5+3 = 8</code>.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 2 5. Relational Operators <pre>#include<stdio.h> int main() { printf("%d \n", 4==4); printf("%d \n", 4<3); printf("%d \n", 3<4); printf("%d \n", 4!=4); return 0; }</pre>	<pre>1 0 1 0</pre>	- Relational operators (<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>) analyze inequalities and matches. - C lacks native Boolean text primitives in core systems; truth states are handled mathematically: <code>0</code> signifies False, while <code>1</code> (or any non-zero value) signifies True.
CHAPTER 2 6. Logical Operators <pre>#include<stdio.h> int main() { printf("%d \n", 3<4 && 3<5); printf("%d \n", 3<4 && 5<4); printf("%d \n", !(3<4 && 3<5)); return 0; }</pre>	<pre>1 0 0</pre>	- Logical operators compound conditional statements: <code>&&</code> (Logical AND) returns true only if every surrounding assertion holds true. <code> </code> (Logical OR) returns true if at least one individual assertion holds true. <code>!</code> (Logical NOT) flips the boolean outcome (e.g., <code>!True</code> becomes <code>0</code>).

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 2 7. Compound Assignment Operators <pre>#include <stdio.h> int main() { int a = 10; a += 10; // a = a + 10 printf("a+10 = %d \n", a); a %= 10; printf("a%%c10 = %d \n", '%', a); return 0; }</pre>	<pre>a+10 = 20 a%10 = 0</pre>	- Compound assignment operators (<code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code>) offer performance-optimized shorthands for updating a variable by operating on its own existing value. Syntax Trick: To output a raw percent character (<code>%</code>) via <code>printf</code> inside running strings, pass it explicitly as a character arg (<code>'%c'</code>) or duplicate it within formatting text (<code>%%</code>).
CHAPTER 3 1. Simple If-Else Conditional <pre>#include<stdio.h> int main() { int age = 19; if(age >= 18) { printf("you are an adult"); } else { printf("you are not an adult"); } return 0; }</pre>	you are an adult	- Introduces logical control branching. - The <code>if</code> block triggers execution when the wrapped boolean condition evaluates to true (non-zero). If it is false (zero), execution cascades straight to the fallback <code>else</code> segment.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 3 2. Odd or Even Checker <pre>#include<stdio.h> int main() { int number; scanf("%d", &number); if(number % 2 == 0) { printf("even"); } else { printf("odd"); } return 0; }</pre>	7 (User input) odd	- Combines relational equations with arithmetic modulo operations inside control loops. - Checks integer divisibility: if a digit divided by 2 leaves a remainder absolute of 0, it satisfies the even block; otherwise, it executes the odd block.
CHAPTER 3 3. Multi-Way Else-If Ladder <pre>#include<stdio.h> int main() { int age = 14; if(age < 12) { printf("child"); } else if(age < 18) { printf("teenager"); } else { printf("adult"); } return 0; }</pre>	teenager	- Demonstrates sequential condition checking using an <code>else if</code> ladder. - The conditions are evaluated from the top down. As soon as a condition is satisfied, its block executes and the rest of the ladder is skipped. If no conditions match, the final <code>else</code> block executes.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 3 4. Ternary Operator Shorthand <pre>#include<stdio.h> int main() { int age = 19; age > 18 ? printf("adult \n") : printf("not adult \n"); return 0; }</pre>	adult	- The Ternary Operator (<code>? : </code>) is a compact, single-line shorthand for simple if-else blocks. Syntax: <code>Condition ? Expression_If_True : Expression_If_False;</code> . Excellent for inline assignments and concise structural branching.
CHAPTER 3 5. Switch Case (Integer & Character Expression) <pre>#include<stdio.h> int main() { char day = 'f'; switch(day) { case 'm' : printf("monday \n"); break; case 'f' : printf("friday \n"); break; default : printf("invalid day \n"); } return 0; }</pre>	friday	<code>switch</code> matches an expression against multiple constant literal cases. It is often cleaner and more efficient than a long if-else ladder. Crucial Concept: A <code>break;</code> statement is required at the end of each case block. Without it, execution will continue into the following cases regardless of whether they match, a behavior known as fall-through.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 4 1. Iterative Control: For, While, & Do-While Loops <pre>#include <stdio.h> int main () { // for loop for(int i=1; i<=3; i++) { printf("%d ", i); } // while loop int j=1; while(j<=3) { printf("%d ", j); j++; } // do-while loop int k=1; do { printf("%d ", k); k++; } while(k<=3); return 0; }</pre>	<pre>1 2 3 1 2 3 1 2 3</pre>	- Compares the three fundamental loops in C: for loop: Best when the total iteration count is known beforehand. It bundles initialization, condition checking, and incrementing together. while loop: An entry-controlled loop that evaluates the condition <i>before</i> entering the loop body. do-while loop: An exit-controlled loop that evaluates the condition <i>after</i> executing the loop body, guaranteeing the block runs at least once.
CHAPTER 5 1. Basic Function Prototype & Architecture <pre>#include<stdio.h> void printHello(); // Declaration/Prototype int main() { printHello(); // Function Call return 0; } void printHello() { // Definition printf("Hello!\n"); }</pre>	Hello!	- Introduces structured, modular programming. Functions require three distinct parts: 1. Prototype/Declaration: Alerts the compiler about the function name, arguments, and return type before it is used. 2. Call: Invokes the function from <code>main()</code> or another function. 3. Definition: Contains the actual code block to be executed. - The <code>void</code> keyword indicates that the function does not return any value.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 5 2. Functions with Parameters & Returns <pre>#include <stdio.h> int calcSquare(int n); int main() { int n = 5; printf("square is : %d", calcSquare(n)); return 0; } int calcSquare(int n) { return n * n; }</pre>	square is : 25	• Demonstrates parameterized data passing. The value of variable <code>n</code> in <code>main()</code> is copied into parameter <code>n</code> of <code>calcSquare()</code>. • The <code>return</code> keyword passes the computed value back to the caller, and the function's return type matches the data type of this value (<code>int</code>).
CHAPTER 5 3. Recursive Function (Factorial) <pre>#include <stdio.h> int factorial(int n); int main() { printf("factorial is : %d", factorial(4)); return 0; } int factorial(int n) { if(n == 0) return 1; // Base Case return factorial(n-1) * n; }</pre>	factorial is : 24	• Demonstrates Recursion, a programming technique where a function calls itself to solve smaller instances of the same problem. • Critical Rule: Every recursive function must have a Base Case (e.g., <code>if(n == 0) return 1;</code>) to stop the execution chain. Without it, the function will call itself indefinitely, causing a stack overflow error.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 6 1. Pointer Syntax & Address Operations <pre>#include<stdio.h> int main() { int age = 22; int *ptr = &age; printf("%d\n", *ptr); printf("%p\n", (void*)&age); printf("%p\n", (void*)ptr); return 0; }</pre>	<pre>22 0x7ffee3b4a8ac (Example hex address) 0x7ffee3b4a8ac</pre>	• A Pointer is a variable that stores the memory address of another variable rather than a direct value. • & is the Address-Of operator, used to retrieve the exact memory location of a variable. • * is the Dereference (Value-At-Address) operator, used to access the value stored at the address a pointer points to. • The %p format specifier is used to print memory addresses in hexadecimal format.
CHAPTER 6 2. Call by Value vs. Call by Reference <pre>#include <stdio.h> void square(int n); void _square(int* n); int main() { int num = 4; square(num); // Value printf("Val: %d\n", num); _square(&num); // Reference printf("Ref: %d\n", num); return 0; } void square(int n) { n = n * n; } void _square(int* n) { *n = *n * *n; }</pre>	<pre>Val: 4 Ref: 16</pre>	• Contrasts the two primary methods for passing arguments to functions: • Call by Value: A copy of the variable's value is passed to the function. Modifying this copy inside the function does not affect the original variable. • Call by Reference: The variable's memory address is passed to the function using a pointer. This allows the function to modify the original variable directly.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 6 3. Code Bug Analysis: Swapping Variables <pre>#include <stdio.h> void _swap(int* a, int* b); int main() { int x = 3, y = 5; _swap(&x, &y); printf("x=%d, y=%d\n", x, y); return 0; } void _swap(int* a, int* b) { int t = *a; *a = *b; *b = *a; // LOGICAL BUG IN SOURCE CODE! }</pre>	x=5, y=5 (Due to code bug)	• Analytical Insight: There is a logical bug in the source file's swap function. Line 12 performs <code>*b = *a;</code>. However, <code>*a</code> was already overwritten by <code>*b</code> on the previous line. As a result, <code>y</code> becomes <code>5</code> instead of <code>3</code>. • To fix this, the value stored in the temporary variable <code>t</code> should be assigned instead: <code>*b = t;</code>. This highlights the importance of carefully managing assignments when working with pointers.
CHAPTER 7 1. Array Syntax & Subscript Indexing <pre>#include <stdio.h> int main() { int marks[3]; marks[0] = 85; marks[1] = 90; marks[2] = 95; printf("Physics: %d, Chem: %d", marks[0], marks[1]); return 0; }</pre>	Physics: 85, Chem: 90	• An Array is a collection of variables of the same data type stored in contiguous memory locations. • Array indexing is zero-based, meaning an array of size <code>n</code> has elements indexed from <code>0</code> to <code>n-1</code>. For example, <code>marks[3]</code> contains elements at slots <code>0</code>, <code>1</code>, and <code>2</code>.

1. INPUTS & SOURCE CODES	2. COMPILED / INTENDED OUTPUTS	3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS
CHAPTER 7 2. Pointer Arithmetic and Distance Scales <pre>#include <stdio.h> int main() { int age = 22, _age = 25; int *ptr = &age, *_ptr = &_age; printf("Ptr address: %u\n", ptr); ptr++; printf("After ptr++: %u\n", ptr); return 0; }</pre>	Ptr address: 2000 After ptr++: 2004 (<i>Increments by 4 bytes</i>)	• Pointer arithmetic operations depend on the data type of the pointer. • Incrementing an integer pointer (<code>ptr++</code>) increases its address by <code>sizeof(int)</code> (typically 4 bytes), moving it to point to the next integer location in memory. • Subtracting two pointers (<code>ptr - _ptr</code>) returns the distance between them in terms of the number of elements, not the number of bytes.
CHAPTER 7 3. Passing Arrays into Function Routines <pre>#include <stdio.h> void printNumbers(int *arr, int n); int main() { int arr[] = {1, 2, 3}; printNumbers(arr, 3); return 0; } void printNumbers(int *arr, int n) { for(int i=0; i<n; i++) { printf("%d ", arr[i]); } }</pre>	1 2 3	• In C, arrays are passed to functions by reference. The array name acts as a pointer to its first element (<code>&arr[0]</code>). • In a function signature, the declarations <code>int *arr</code> and <code>int arr[]</code> are equivalent. Both receive the starting memory address of the array.

1. INPUTS & SOURCE CODES

2. COMPILED / INTENDED OUTPUTS

3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS

CHAPTER 8

1. Strings, Buffer Inputs, and String Library Utilities

```
#include <stdio.h>
#include <string.h>
int main() {
    char name[] = "Shradha";
    char fullName[40];

    // Safe multi-word string read
    fgets(fullName, 40, stdin);
    puts(fullName);

    printf("Length: %d", (int)strlen(name));
    return 0;
}
```

```
John Doe (User input)
John Doe
Length: 7
```

- A **String** in C is an array of characters terminated by a null character (`\0`).
- **Input Handling Differences:** `scanf("%s")` stops reading input when it encounters whitespace. In contrast, `fgets(buffer, size, stdin)` safely reads multi-word strings with spaces up to a specified size limit. `puts()` prints a string followed by a newline.
- Requires the `<string.h>` library, which provides useful string manipulation functions:
 - `strlen()` : Computes the length of a string, excluding the null terminator.
 - `strcpy(dest, src)` : Copies the source string to the destination buffer.
 - `strcat(dest, src)` : Appends the source string to the end of the destination string.
 - `strcmp(s1, s2)` : Compares two strings lexicographically based on their characters' ASCII values.

CHAPTER 9

1. Structures, Typedef Aliasing, & Structural Pointers

```
#include <stdio.h>
#include <string.h>
struct student {
    char name[100]; int roll; float cgpa;
};
typedef struct Vector { int x; int y; } vec;
int main() {
    struct student s1;
    strcpy(s1.name, "Shradha"); s1.roll = 64;
    struct student *ptr = &s1;
    printf("Name via arrow: %s\n", ptr->name);
    return 0;
}
```

```
Name via arrow: Shradha
```

- A **Structure** (`struct`) is a user-defined data type that groups related variables of different data types under a single name.
- Structure members are accessed using the dot operator (`.`) for standard structure variables (e.g., `s1.roll`). The arrow operator (`->`) is used when accessing members through a structure pointer (e.g., `ptr->roll` , which is equivalent to `(*ptr).roll`).
- By default, structures are passed to functions **by value**, meaning the function receives a copy of the structure and modifications do not affect the original variable.
- The `typedef` keyword creates a shorter, convenient alias for a structure type (e.g., `vec` instead of `struct Vector`).

1. INPUTS & SOURCE CODES

2. COMPILED / INTENDED OUTPUTS

3. TECHNICAL DETAILS & SYNTACTIC ANALYSIS

CHAPTER 10

1. File I/O Operations (Read, Write, and Streams)

```
#include <stdio.h>
int main() {
    FILE *fptr = fopen("NewFile.txt", "w");
    if(fptr != NULL) {
        fputc('A', fptr);
        fprintf(fptr, "Pple");
        fclose(fptr);
    }
    return 0;
}
```

*(Creates or overwrites
'NewFile.txt' on disk
containing 'Apple')*

- File operations in C require a specialized **FILE** pointer handle.
- **fopen('filename', 'mode')** opens a file stream. Common modes include **'r'** for reading and **'w'** for writing (which overwrites an existing file or creates a new one). Always check if the pointer is **NULL** to ensure the file opened successfully.
- **fgetc()** and **fputc()** are used to read and write single characters, respectively.
- The **EOF** (End Of File) constant indicates that the end of a file stream has been reached during sequential reading.
- **Best Practice:** Always close open file streams using **fclose(fptr)** to ensure cached data is written to disk and system resources are freed.

CHAPTER 11

1. Dynamic Memory Allocation (Heap Management)

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *ptr = (int *) calloc(3, sizeof(int));
    if(ptr != NULL) {
        printf("Initial element: %d\n", ptr[0]);
        free(ptr); // Prevent memory leaks
    }
    return 0;
}
```

Initial element: 0

- **Dynamic Memory Allocation (DMA)** allows a program to request and allocate memory blocks from the heap at runtime, rather than relying on compile-time stack allocation.
- **sizeof()** is a built-in operator that returns the size of a data type or variable in bytes.
- **calloc(num, size)** allocates a contiguous block of memory and automatically initializes all bytes to **0**. In contrast, **malloc()** allocates uninitialized memory, leaving unpredictable garbage values.
- **Memory Management:** Every dynamically allocated memory block must be explicitly deallocated using **free(ptr)** when it is no longer needed. Failing to do so causes a **Memory Leak**, which consumes system memory and can lead to application crashes.